Atividade de Revisão
Semestre: 4º Semestre - Curso de Desenvolvimento de Software Multiplataforma
Disciplina: Integração e Entrega Contínua (IEC)
Referência: Aulas 02 a 06

Tema: Segurança Digital em CI/CD

QUESTÃO 1 - Uma equipe desenvolve um sistema crítico de monitoramento ambiental. O pipeline atual é:

```
on:
  push:
    branches: [main]
```

Após um incidente de segurança, identificou-se que código vulnerável foi direto para produção. Qual alteração é MAIS adequada para mitigar esse risco?

a) Adicionar logs detalhados
b) Alterar para pull_request e exigir validação antes do merge
c) Remover a branch main
d) Executar apenas testes manuais
e) Criar mais commits

QUESTÃO 2 - Considere o trecho:

```
- run: npm install
```

Durante uma auditoria, foi identificado risco de supply chain attack.
Qual alternativa melhor mitiga esse risco?

a) Substituir por npm start
b) Usar npm ci com package-lock.json
c) Executar o comando apenas localmente
d) Remover dependências
e) Usar apenas GitHub Actions

QUESTÃO 3 - Um pipeline executa:

```
- run: npm test
- run: npm run build
```

Em um incidente, descobriu-se que o build estava sendo gerado mesmo com testes falhando. Qual explicação é MAIS provável?

a) YAML não suporta testes
b) Falta de dependência entre etapas
c) npm ignora erros automaticamente
d) build executa primeiro
e) GitHub não executa scripts

QUESTÃO 4 - Um desenvolvedor adicionou o arquivo .env ao repositório para "resolver erro no pipeline".
Qual é o PRINCIPAL risco dessa ação?
a) aumento do tamanho do projeto
b) perda de performance
c) exposição de credenciais sensíveis
d) falha de build
e) erro de sintaxe

QUESTÃO 5 - Considere:

```
env:
  API_KEY: "123456"
```

Qual é a melhor prática de segurança?
a) manter a chave no código
b) usar variáveis locais
c) usar GitHub Secrets
d) remover a API
e) colocar em README

QUESTÃO 6 - Em um sistema IoT, falhas de lint estavam sendo ignoradas.
Qual impacto MAIS crítico isso pode causar?
a) aumento de performance
b) falhas visuais
c) introdução de vulnerabilidades no código
d) perda de logs
e) falha no Git

QUESTÃO 7 - Pipeline:

```
on:
  pull_request:
```

Qual vantagem de segurança essa configuração oferece?
a) elimina commits
b) valida código antes de entrar na main
c) impede testes
d) substitui logs
e) remove branches

QUESTÃO 8 - Um projeto NÃO possui logs no pipeline.
Qual é o maior problema?
a) código maior
b) execução lenta
c) impossibilidade de auditoria e diagnóstico
d) erro de sintaxe
e) falha de Git

QUESTÃO 9 - Considere:

```
- run: npm audit
```

Qual limitação torna esse uso insuficiente?
a) não roda no GitHub
b) não detecta vulnerabilidades
c) não falha o pipeline automaticamente
d) não usa YAML
e) não executa testes

QUESTÃO 10 - Um pipeline permite push direto na main.
Qual prática resolve corretamente?
a) apagar a main
b) criar branch protection rules
c) remover CI
d) usar apenas commits locais
e) desativar Actions

QUESTÃO 11 - Considere:

```
jobs:
  build:
    runs-on: ubuntu-latest
```

Qual risco está associado a ambientes não controlados?
a) execução lenta
b) inconsistência entre ambientes
c) perda de código
d) erro de YAML
e) falta de logs

QUESTÃO 12 - Um sistema executa deploy automático sem testes.
Qual risco é MAIS crítico?
a) código maior
b) deploy de código vulnerável
c) perda de commits
d) erro de sintaxe
e) falta de branches

QUESTÃO 13 - Sobre automação:
Qual afirmação é correta no contexto de segurança?
a) automação elimina erros humanos
b) automação reduz risco, mas não elimina falhas
c) automação substitui testes
d) automação elimina vulnerabilidades
e) automação impede ataques

QUESTÃO 14 - Considere:

- run: eslint .

Qual o papel dessa etapa no pipeline?
a) executar testes
b) detectar problemas de qualidade e possíveis falhas
c) instalar dependências
d) gerar build
e) fazer deploy

QUESTÃO 15 - Um pipeline não possui monitoramento contínuo.
Qual consequência?
a) melhoria de desempenho
b) falhas não detectadas em produção
c) redução de código
d) aumento de commits
e) eliminação de bugs

QUESTÃO 16 - Considere um sistema crítico (ex: alertas ambientais).
Por que CI é essencial?
a) aumenta código
b) valida continuamente a integridade do sistema
c) elimina deploy

d) substitui logs
e) impede commits

QUESTÃO 17 - Um pipeline falhou, mas não há logs.
Qual é o erro de arquitetura?
a) ausência de CI
b) ausência de observabilidade
c) uso de Git
d) uso de YAML
e) uso de npm

QUESTÃO 18 - Sobre observabilidade:
Qual alternativa melhor define o conceito?
a) apagar logs
b) esconder falhas
c) entender o comportamento do sistema via dados
d) executar testes
e) impedir deploy

QUESTÃO 19 - Um desenvolvedor ignora o ESLint.
Qual risco?
a) código menor
b) vulnerabilidades e inconsistências
c) falha no Git
d) erro de YAML
e) perda de logs

QUESTÃO 20 - Pipeline seguro deve:
a) executar apenas build
b) ignorar erros
c) integrar testes, auditoria e validação automática
d) evitar CI
e) eliminar logs